

Cheat-sheet di Intelligenza Artificiale I

Formule, definizioni essenziali, condizioni e trappole d'esame

1. Agenti razionali

Un agente razionale sceglie l'azione che massimizza il valore atteso della misura di prestazione, date le percezioni e le conoscenze disponibili. PEAS significa **P**erformance, **E**nvironment, **A**ctuators, **S**ensors.

- Ambiente: osservabile/parzialmente osservabile; deterministico/stocastico; episodico/sequenziale; statico/dinamico; discreto/continuo; singolo/multi-agente.
- Reflex: usa la percezione corrente. Reflex con modello: mantiene uno stato interno. Goal-based: pianifica verso un obiettivo. Utility-based: confronta esiti con gradi diversi di desiderabilità.

$$\mathbb{E}[U(X)] = \sum_x P(X=x) U(x)$$

L'utilità attesa tiene distinti probabilità degli esiti e preferenze dell'agente. Valore monetario e utilità non coincidono necessariamente.

2. Problemi di ricerca

$$P = (S, s_I, G, A, f, c)$$

- S : stati; s_I : stato iniziale; G : goal test o insieme dei goal.
- $A(s)$: azioni applicabili; $f(s, a)$: transizione; $c(s, a, s')$: costo.
- L'albero di ricerca contiene storie di azioni e può duplicare uno stato; il grafo degli stati descrive il problema.

$$g(n) = \sum_{i=0}^{k-1} c(s_i, a_i, s_{i+1}) \quad 1 + b + \dots + b^d = \frac{b^{d+1} - 1}{b - 1} = O(b^d)$$

DFS, BFS, UCS, Greedy e A*

Algoritmo	Priorità FRONTIER	Goal test	Proprietà chiave
DFS	nodo più profondo, LIFO	generazione	spazio $O(d)$; non ottima
BFS	nodo meno profondo, FIFO	generazione	completa; ottima con costi uniformi
UCS	minimo $g(n)$	estrazione	completa e ottima con costi positivi
Greedy	minimo $h(n)$	estrazione	spesso rapida, non ottima
A*	minimo $f(n)$	estrazione	ottima sotto le condizioni sull'euristica

Per UCS e A* non fermarti quando il goal viene generato: un cammino più economico potrebbe essere ancora in frontiera. EQL marca i nodi accodati; EXL quelli estratti/espansi.

A* ed euristiche

$$f(n) = g(n) + h(n)$$

Ammissibilità e consistenza:

$$0 \leq h(n) \leq h^*(n) \quad \forall n, \quad h(s) \leq c(s, a, s') + h(s')$$

- Ammissibile: h non sovrastima il costo ottimo residuo.
- Consistente: soddisfa la disuguaglianza triangolare e rende f non decrescente lungo i cammini.
- Tree search: l'ammissibilità basta per l'ottimalità. Graph search con EXL: nel modello del corso si richiede consistenza.
- Se $h_2(n) \geq h_1(n)$ ovunque, h_2 domina h_1 .

$$h_3(n) = \max\{h_1(n), h_2(n)\}$$

Se h_1 e h_2 sono consistenti, anche il loro massimo lo è. Un rilassamento rimuove vincoli: il costo ottimo del problema rilassato fornisce un'euristica consistente per l'originale.

Focal Search

$$\begin{aligned} n_{\text{best}} &= \arg \min_{n \in F} f(n), \\ \text{FOCAL} &= \{n \in F \mid f(n) \leq \omega f(n_{\text{best}})\}, \quad \omega \geq 1, \\ n_{\text{next}} &= \arg \min_{n \in \text{FOCAL}} \hat{h}_F(n). \end{aligned}$$

Quando viene estratto un goal e :

$$g(e) = f(e) \leq \omega f(n_{\text{best}}) \leq \omega \text{OPT}$$

La soluzione è bounded-suboptimal di fattore ω . Con $\omega = 1$ si recupera il comportamento di A^* .

3. Giochi

Un gioco specifica stati, giocatore al turno, azioni, transizione, test terminale e utilità. Informazione *completa* riguarda regole e payoff; informazione *perfetta* riguarda l'osservabilità di storia e stato.

Minimax

$$V(s) = \begin{cases} u_{MAX}(s), & T(s), \\ \max_{a \in A(s)} V(f(s, a)), & I(s) = MAX, \\ \min_{a \in A(s)} V(f(s, a)), & I(s) = MIN. \end{cases}$$

Tempo $O(b^d)$ e spazio $O(d)$ con visita DFS. Contro un avversario razionale fornisce la strategia ottima.

Potatura alpha-beta

- Nodo MAX: $v \leftarrow \max(v, V_{figlio})$, $\alpha \leftarrow \max(\alpha, v)$; pota se $v \geq \beta$.
- Nodo MIN: $v \leftarrow \min(v, V_{figlio})$, $\beta \leftarrow \min(\beta, v)$; pota se $v \leq \alpha$.
- Non cambia il risultato di Minimax. Con ordinamento perfetto: $O(b^{d/2})$.

Da ricordare: Annota α e β entrando nel nodo; aggiornali solo dopo il ritorno di un figlio. I fratelli ereditano i bound del padre, non quelli locali del fratello.

Valutazione al cutoff:

$$v(s) = w_1 f_1(s) + \dots + w_n f_n(s)$$

La quiescent search evita cutoff in stati tatticamente instabili. Le transposition table riusano risultati di stati raggiunti tramite cammini diversi.

Expectimax e MCTS

$$V_{CHANCE}(s) = \sum_a P(a \mid s) V(f(s, a))$$

$$\hat{w}_i = \frac{W_i}{N_i}, \quad \text{UCB}_i = \hat{w}_i + c\sqrt{\frac{\log N}{N_i}}$$

Il primo termine di UCB sfrutta le azioni promettenti; il secondo favorisce l'esplorazione.

4. Constraint Satisfaction Problems

$$\text{CSP} = (X, D, C), \quad X = \{X_1, \dots, X_n\}, \quad D = \{D_1, \dots, D_n\}$$

Una soluzione è un assegnamento completo che soddisfa tutti i vincoli.

- Node consistency: ogni valore soddisfa i vincoli unari.
- Arc consistency di $X_i \rightarrow X_j$: ogni valore di D_i ha almeno un supporto in D_j .
- Path consistency: ogni coppia consistente ha supporto in una terza variabile.

$$D_i \leftarrow D_i \setminus \{x \in D_i \mid \nexists y \in D_j : C_{ij}(x, y)\}$$

AC-3 ripete REVISE sugli archi orientati; se D_i cambia reinserisce gli archi (X_k, X_i) dei vicini. Un dominio vuoto implica fallimento.

Backtracking e ricerca locale

- MRV: variabile con meno valori rimasti.
- Degree heuristic: a parità, variabile che vincola più variabili non assegnate.
- LCV: prova prima il valore che elimina meno possibilità ai vicini.
- Forward checking propaga ai vicini; MAC mantiene l'arc consistency dopo ogni assegnamento.
- Min-Conflicts sceglie una variabile in conflitto e il valore che minimizza i vincoli violati.

5. Markov Decision Processes

$$\text{MDP} = (S, A, P, R, \gamma)$$

Proprietà di Markov e ritorno scontato:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t),$$

$$G_0 = \sum_{t=0}^H \gamma^t R_t.$$

Con orizzonte infinito, $0 \leq \gamma < 1$ e $|R_t| \leq R_{max}$:

$$|G_0| \leq \frac{R_{max}}{1 - \gamma}, \quad \pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[G_0]$$

Equazioni di Bellman

$$Q^*(s, a) = \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V^*(s')],$$

$$V^*(s) = \max_a Q^*(s, a),$$

$$\pi^*(s) = \arg \max_a \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V^*(s')].$$

Value Iteration

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')]$$

$$\delta_\infty = \max_s |V_{k+1}(s) - V_k(s)| \leq \varepsilon$$

Negli aggiornamenti sincroni usa soltanto V_k a destra e assegna simultaneamente i valori di V_{k+1} .

Policy Iteration

$$V^\pi(s) = \sum_{s'} P(s' | s, \pi(s)) [R(s, \pi(s), s') + \gamma V^\pi(s')],$$

$$\pi'(s) = \arg \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^\pi(s')].$$

Il ciclo è: policy evaluation, policy improvement, ripetizione finché la policy non cambia.

6. Reinforcement Learning passivo

Nel RL passivo la policy è fissata; bisogna stimarne il valore da esperienze campionate.

Adaptive Dynamic Programming

$$\hat{P}(s' | s, a) = \frac{N(s, a, s')}{N(s, a)}, \quad \hat{R}(s, a, s') = \frac{\sum r}{N(s, a, s')}$$

ADP costruisce un modello empirico e risolve le equazioni di Bellman.

Direct Utility Estimation e Temporal Difference

$$V(s) \leftarrow \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_i$$

$$V(s) \leftarrow V(s) + \alpha \underbrace{[R + \gamma V(s') - V(s)]}_{\delta, \text{ TD error}}$$

Media incrementale e condizioni classiche sulla learning rate:

$$\bar{x}_n = \bar{x}_{n-1} + \frac{1}{n}(x_n - \bar{x}_{n-1}), \quad \sum_t \alpha_t = \infty, \quad \sum_t \alpha_t^2 < \infty$$

7. Reinforcement Learning attivo

Q-Learning: off-policy

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Il target usa l'azione greedy, anche se la policy di comportamento sta esplorando.

SARSA: on-policy

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma Q(s', a') - Q(s, a)]$$

Il target usa l'azione a' realmente scelta dalla policy corrente.

Esplorazione

$$P(a | s) = \begin{cases} 1 - \varepsilon + \varepsilon/|A(s)|, & a \in \arg \max_b Q(s, b), \\ \varepsilon/|A(s)|, & \text{altrimenti.} \end{cases}$$

Per SARSA serve tipicamente GLIE: ogni coppia stato-azione viene visitata infinite volte, ma la policy diventa greedy nel limite.

Robustezza e approssimazione

$$\pi_{Bayes}^* = \arg \max_{\pi} \sum_i P(m_i) V_i^{\pi}, \quad \pi_{robust}^* = \arg \max_{\pi} \min_i V_i^{\pi}$$

$$Q_{\theta}(s, a) = \theta_0 + \sum_i \theta_i f_i(s, a),$$

$$\theta_i \leftarrow \theta_i + \alpha \left[R + \gamma \max_{a'} Q_{\theta}(s', a') - Q_{\theta}(s, a) \right] f_i(s, a).$$

8. Machine Learning supervisionato

$$\mathcal{T} = \{(\mathbf{x}_1, z_1), \dots, (\mathbf{x}_N, z_N)\}, \quad z_i = f(\mathbf{x}_i), \quad h \approx f$$

Regressione lineare

$$h_{\theta}(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \dots + \theta_n x_n = \boldsymbol{\theta}^T \tilde{\mathbf{x}},$$

$$\text{MSE}(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N [z_i - h_{\theta}(\mathbf{x}_i)]^2.$$

Caso univariato:

$$\theta_1 = \frac{\sum_i (x_i - \bar{x})(z_i - \bar{z})}{\sum_i (x_i - \bar{x})^2}, \quad \theta_0 = \bar{z} - \theta_1 \bar{x}$$

Update SGD con loss $\frac{1}{2}(z - h)^2$:

$$\theta_j \leftarrow \theta_j + \alpha [z_i - h_{\theta}(\mathbf{x}_i)] x_{i,j}$$

Regressione logistica

$$\sigma(t) = \frac{1}{1 + e^{-t}}, \quad h(\mathbf{x}) = \sigma(\boldsymbol{\theta}^T \tilde{\mathbf{x}})$$

Alberi di decisione

$$\begin{aligned}\mathcal{H}(Y) &= - \sum_c p(c) \log_2 p(c), \\ \mathcal{H}(Y | X) &= \sum_v \frac{|\mathcal{T}_v|}{|\mathcal{T}|} \mathcal{H}(Y | \mathcal{T}_v), \\ \text{IG}(Y, X) &= \mathcal{H}(Y) - \mathcal{H}(Y | X).\end{aligned}$$

Si sceglie lo split con information gain massimo, cioè con massima riduzione dell'incertezza media.

K-Nearest Neighbors

$$L_p(\mathbf{x}, \mathbf{y}) = \left(\sum_i |x_i - y_i|^p \right)^{1/p}$$

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = - \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}, \quad L \approx \left(\frac{k}{n} \right)^{1/d}$$

Il secondo risultato esprime la curse of dimensionality: al crescere di d , una frazione fissata di vicini occupa una regione sempre più estesa.

Feature engineering

$$\hat{x}_i = \frac{x_i - \min_h x_i^{(h)}}{\max_h x_i^{(h)} - \min_h x_i^{(h)}}, \quad \hat{x}_i = \frac{x_i - \mu_i}{\sigma_i}$$

Le statistiche di normalizzazione si stimano solo sul training set. Per feature nominali usa one-hot encoding, non una numerazione arbitraria.

9. Reti neurali, metodologia e metriche

Unità e attivazioni

$$y = g \left(w_0 + \sum_i w_i x_i \right)$$

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}, \quad \text{ReLU}(x) = \max(0, x), \quad \text{softplus}(x) = \log(1 + e^x), \quad \tanh(x) = \frac{e^{2x} - 1}{e^{2x} + 1}$$

Senza attivazioni non lineari, la composizione di più layer resta lineare. Backpropagation applica la chain rule e riutilizza i sottocalcoli. Prodotti ripetuti di derivate piccole causano vanishing gradients.

Split dei dati e generalizzazione

- Training: apprendimento dei parametri.
- Validation: scelta di modello e iperparametri.
- Test: stima finale, da usare una sola volta.
- Underfitting: bias alto ed errore alto anche sul training.
- Overfitting: varianza alta; training buono, validation/test scarsi.

Metriche

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad \text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{MSE}_{test} = \frac{1}{N_{test}} \sum_i [z_i - h(\mathbf{x}_i)]^2$$

L'accuracy può essere fuorviante con classi fortemente sbilanciate.

10. Checklist rapida da esame

1. Scrivi la formula e definisci ogni simbolo.
2. Indica le ipotesi: costi positivi, consistenza, $\gamma < 1$, visite infinite, ecc.
3. Distingui garanzia teorica e comportamento pratico.
4. Nei calcoli iterativi separa chiaramente valori vecchi e nuovi.
5. Controlla normalizzazione delle probabilità e segni di reward e costi.
6. Concludi con il limite principale del metodo.

Da ricordare: Formula + significato dei simboli + ipotesi + garanzia + limite pratico: questa struttura copre quasi ogni domanda teorica breve.